SQL Parsing Application

Project By: Viviana Alba, Kayla Hoffman

1.0 Introduction: Project and Application Description

Modern data processing workflows frequently rely on high-level Python libraries such as Pandas, which provides convenient abstractions for loading, manipulating, and analyzing structured and semi-structured data. These abstractions allow users to perform operations such as filtering, projection, grouping, aggregation, and join with minimal effort, allowing them to become pioneers in many analytical and data-driven applications. While these libraries are quick and easy to use, they hide substantial underlying complexity, which often serves as foundational knowledge for programmers of all levels.

The objective of this project is to explore what happens under the hood of these popular, high-level tools by re-implementing core data-processing functionality from first principles. Through this process, we investigate the scaling and parsing challenges that modern Python libraries so gracefully resolve. Instead of relying on Pandas for analytical logic, we developed a custom system capable of parsing CSV files, storing data in memory, and supporting essential relational operations such as filtering, sorting, limiting, projection, grouping, aggregation, and joining. As a response to the exponentially growing field of data science, scalability was addressed in this application through caching, pagination, and chunked reading, which allows data to be loaded and filtered incrementally when the dataset is too large to fit entirely in main memory. In addition to functional re-implementation, this project also incorporates an interactive, user-friendly dashboard through Streamlit that demonstrates how these low-level operations can be used to apply custom transformations on real-world datasets.

2.0 Dataset Descriptions

To demonstrate the functionality of our application, we used a *Global Country Information*[1] dataset from Kaggle that was published in 2023 and a *United States Food Imports*[3] dataset from the U.S. Department of Agriculture that was last updated in January 2025. The goal of our application is to explore how trends in food imports compare between countries. The *Global Country Information* dataset provides country-level demographic, economic, health, education, and environmental indicators for all countries worldwide. It contains 228 columns that consist of different countries and 35 features that include data such as “Country”, "Agricultural Land (%)”, “GDP”, and “Population” to name a few. On the other hand, the *United States Food Imports* dataset provides annual information on the value and origin of food and beverage products imported into the United States. It is organized by food group and country and reflects long-term trends in consumer demand, seasonality, and globalized food production. With over 20 years of historical data, the dataset supports analysis of import growth patterns, trade relationships, and changes in food sourcing over time. It contains 19,839 columns and 7 features that include data such as “Commodity”, “Country”, and “FoodValue.”

Both datasets share the common attribute "Country," which enables the application to perform inner and left join operations across the two sources. This shared feature allows users to combine country-level socioeconomic indicators with food import statistics, facilitating comparative analyses that connect national characteristics with patterns in U.S. food trade. By joining these datasets, the application supports exploratory queries such as examining how economic indicators relate to the types and values of food products imported from different countries.

3.0 Methodology

3.1 Overview and Pipeline

This project replicates core SQL-style data processing operations using custom Python functions. The application is built around a modular pipeline that allows users to apply operations incrementally or execute a complete workflow in a single step. Data is first loaded using a custom CSV parser that stores the data as lists of dictionaries, where each dictionary represents a row and each key corresponds to a column attribute. Once loaded, users can apply a sequence of transformations through an easy-to-use, accessible application interface. These include filtering, which selects rows based on column-level predicates; projection, which restricts the dataset to a subset of columns; and sorting, which orders records in ascending or descending order based on numeric or string values. The application also supports grouping and aggregation, enabling users to group records by a selected attribute and compute summary statistics such as count, sum, average, minimum, or maximum. To support data integration, inner and left join operations are implemented using shared keys between datasets, allowing records from multiple sources to be combined for comparative analysis. The overall processing pipeline, shown in **Figure 1.0,** follows a logical execution order consistent with SQL query semantics. Users can execute each step independently or selectively choose which operations to apply, allowing for both simple exploratory queries and more complex multi-step analyses.

Figure 1.0: Application Query Execution Pipeline

3.2 Data Parsing

Parsing serves as a foundational component of the application and supports all downstream data processing operations. The custom parsing pipeline is organized across three levels, row level, value level, and object level, each of which is responsible for progressively interpreting raw CSV input into structured Python objects. At the row level, the parse_csv_line function processes an entire CSV row, correctly splitting it into individual fields while handling quoted values and escaped quotation marks. At the value level, the parse_field function interprets each extracted field and converts it into the appropriate

Python data type. Finally, at the object level, the parse_object function is invoked when a field represents a structured, object-like string.

The parsing process starts with parse_csv_line, which acts as the entry point for a raw CSV input. This function iterates character by character through each row, breaking the data into cells correctly (respecting quotes and escaped quotes) and then transferring each cell to parse_field, which may utilize parse_object. The most important aspect of the parse_csv_line function is tracking whether it is currently inside a quoted field. If the function sees a double quote (“) and is already inside quotes while the next character is also a double quote, the result is the function treats this as an escaped quote and appends one double quote to the field. On the other hand, if a single comma is detected outside the quotes, the function treats this as the end of the current CSV field and sends the accumulated field to parse_field(field) and appends the parsed value to fields.

The next step involves a parse_object(field) function that is a lightweight custom parser for text-based objects, similar in structure to parsing JSON strings. This function starts with removing surrounding braces and the result is the raw text inside the braces. The next step is initializing an empty dictionary that will store the key-value pairs that the function extracts. The function also accounts for empty content and immediately returns an empty dictionary, if this is the case. Then each key-value pair is split in order to be properly parsed. The parsing of each key-value pair happens through a series of steps. First, the pair needs to contain a colon which is then used to split the key and value into two parts. The key is then stripped of whitespace and either single or double quotes. Lastly, the value is passed to another helper function parse_field(value) which handles different data types before the parsed key-value is added to the dictionary.

The parse_field function serves as the core value interpreter within the parsing pipeline. It normalizes numeric formats, preserves special tokens, and recursively invokes parse_object when nested structures are detected. By converting raw string values into properly typed Python objects such as integers, floating-point numbers, or dictionaries, parse_field ensures that downstream operations operate on consistent and meaningful data types. Together, these three parsing layers form a robust and extensible pipeline for transforming raw CSV input into structured data suitable for analysis.

3.3 Read CSV and Chunked Reading

The read_csv function and chunked reading function serve as a primary example of a two-level I/O pipeline where chunked_csv_reader streams rows from disk in batches, and read_csv sits on top to normalize, structure, and optionally tabularize the data from downstream operations such as filter/join. In full-file mode, which occurs when the chunk size is None or set to zero, the entire CSV file is loaded into memory at once. The first line is parsed using parse_csv_line to extract and normalize column headers. Each subsequent non-empty line is then parsed into raw values, which are automatically passed through

parse_field. To ensure structural consistency, the resulting value list is padded or truncated to match the number of headers. Each value is further normalized based on its type: string values are stripped of surrounding whitespace and standardized to support numeric casting (e.g., replacing commas with periods), while non-string values are retained as-is. If numeric conversion fails, values are preserved as lowercase strings.

When the chunk size is greater than zero, chunked mode is activated. In this mode, chunked_csv_reader iterates through the file and yields subsets of rows as dictionaries, which allows the application to process large datasets without loading the entire file into memory. For each chunk, read_csv performs additional normalization of keys and values to ensure consistency with the non-chunked processing path. The primary distinction between these two modes lies in where normalization occurs: in full-file mode, raw lines are first parsed and then normalized, whereas in chunked mode, structured row dictionaries are streamed and subsequently refined.

3.4 Join

The join function is designed to mimic SQL-style inner and left join on column. An inner join returns only rows with matching join key values in both tables, whereas a left join preserves all rows from the left table while appending matching values from the right table when available and inserting null placeholders otherwise. The user-facing entry point for this functionality is the read_and_join function, which loads two CSV files using the custom read_csv parser and dispatches the join operation based on the selected join type. The core join logic is implemented in the inner_join and left_join functions. The inner_join function returns only rows with matching join key values in both tables, while the left_join function preserves all rows from the left table and appends matching values from the right table when available, inserting null placeholders when no match exists. Both functions support single-column and composite join keys by normalizing the join attributes into tuple form.

To add robustness, the helper functions merge_rows and join_rows_as_string ensure that joined records are combined without key collisions and can be reliably converted into a clean string representation. The merge_rows function preserves overlapping column names by copying all key–value pairs from the left table row and appending values from the right table only when no conflict exists, or by suffixing keys when necessary. The join_rows_as_string function further supports output formatting by extracting row values in key order and concatenating them using a specified separator, enabling convenient serialization of joined records for display.

3.5 Projection

Projection replicates the SQL select statement on in memory data by taking a list of row dictionaries and returning new rows that contain only the specified columns. This is a column level filter and does not change the number of rows, only the set of columns. The user can employ projection for

single column or multi column projection. In our pipeline, projection allows the user to reduce data to only the fields needed for downstream operations, visualization, or export. In addition, projection enforces that only valid, existing columns are requested, improving robustness within the dashboard.

3.6 Filter

Filtering was implemented through a custom row-wise evaluation algorithm that selects records based on user-defined predicates. The filter_data function accepts six inputs: the dataset to be filtered, the column to evaluate, the comparison operator, the user-defined value against which each row is compared, and two optional parameters that enable chunked processing and specify the chunk size. The algorithm first determines the data type of the user-provided value by attempting to convert it to a float. If the conversion is successful, the filter is treated as numeric; otherwise, the value is normalized as a lowercase, whitespace-trimmed string to support case-insensitive string comparisons. This allows for the user entry " Canada" to match the data's instance of "canada." Numeric comparisons support the full set of relational operators (>, <, >=, <=, ==, !=), while string comparisons are intentionally limited to equality and inequality to maintain semantic consistency. To improve readability and usability, comparison operators were mapped to user-friendly labels such as "greater than" and "less than" during the implementation of the UI dashboard. Additionally, scalability requirements were addressed in filtering by optionally allowing for chunked execution where each chunk of data is filtered independently using the same predicate logic and the results are accumulated into a final output list, allowing the application to filter datasets that are too large to be fully loaded into main memory.

3.7 Sort

Sorting was implemented using a custom sort_data function that accepts three inputs: the dataset, the column to be sorted, and the desired order ("asc" for ascending or "desc" for descending). The sorting process begins by extracting comparison values from each row using a helper function. Similar to the filter function's logic, this function attempts to convert column values to floats to support numeric sorting. If conversion fails, the original string value is retained. This approach allows for consistent ordering across both numeric and categorical columns while preserving the original data structure.

The core sorting logic is implemented using the quicksort algorithm, which is a divide-and-conquer sorting technique that operates by selecting a pivot element and partitioning the dataset into three subsets: elements with values less than the pivot, elements equal to the pivot, and elements greater than the pivot. This process is illustrated in **Figure 1.1**, where the initial pivot value (5, highlighted in blue) partitions the dataset, and the resulting subarrays are

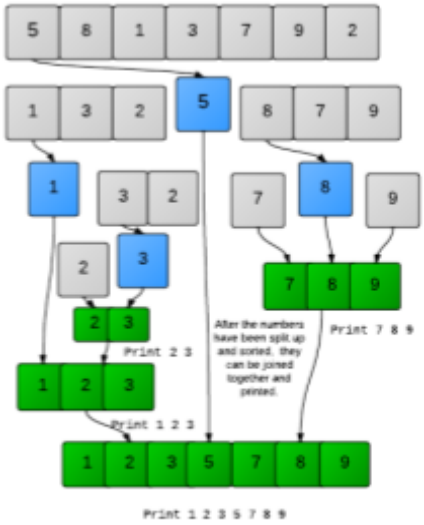

Figure 1.1 QuickSort Partitioning & Merging Algorithm

recursively sorted and subsequently concatenated to produce the final ordered output.[2] This algorithm sorts the data in ascending order, but the function optionally reverses the result if a descending order is requested by the user. The quicksort algorithm was chosen due to its efficiency and conceptual simplicity. Its recursive structure and partition-based logic are straightforward to implement and allow for sorting to be performed directly in-memory. In addition, it has an average-case time complexity of O(n log n), making it well-suited for interactive applications where responsiveness is key.

3.8 Group By and Aggregate

The group and aggregate component of the application is designed to mirror the SQL GROUP BY clause combined with aggregation functions, while accounting for real-world data irregularities such as missing values and mixed data types. To ensure consistency in grouping behavior, a preprocessing step is applied through the clean_key_value function, which normalizes missing data (e.g., empty strings and <NA> values) by consolidating them into a single "Unknown" group. This prevents missing values from fragmenting into multiple singleton groups while preserving their analytical relevance.

The grouping and aggregation logic is implemented in a modular manner. The base group_by function performs raw grouping without aggregation by iterating over each row and organizing records into a dictionary keyed by the selected grouping attribute. Building on this structure, the group_by_aggregate function applies aggregation by first invoking the base grouping function and then computing a user-selected aggregate function (count, sum, avg, min, or max) over the specified column within each group. During aggregation, values are parsed and normalized using the custom parse_field function to ensure robust numeric handling. Aggregation values are first parsed and converted to floating-point numbers when possible; if conversion fails, the original values are retained, allowing aggregation functions such as count to execute without raising runtime errors. This separation allows the application to support both simple grouping operations and aggregated summaries using a shared foundation. Additionally, the implementation supports nested or multi-level grouping through a recursive strategy, enabling hierarchical summaries and providing a foundation for future extensions.

3.9 Limit

The application also includes a custom limit function, which operates on the processed dataset and allows users to view a truncated but representative subset of the final results without altering the underlying data transformations. This implementation was selected for its simplicity, efficiency, and clarity. The limit_data function accepts two parameters: the processed dataset and an integer value specifying the maximum number of rows to return. If no limit value is provided, the function returns the full dataset. If the specified limit is zero or negative, an empty result set is returned to prevent invalid indexing and to provide predictable output for edge cases. Otherwise, the function returns the first n rows of the dataset using Python’s list slicing mechanism.

3.10 Additional Scalability Features: Table Pagination and Caching

During initial testing, the application loaded entire datasets in one page, resulting in reduced responsiveness and a potential for "scroll fatigue" from the user when handling larger datasets. As a response to this issue, the application incorporates table pagination and data caching mechanisms to reduce memory overhead, minimize rendering costs, and improve application responsiveness.

Table pagination is implemented through a custom paginate_table function, which displays the processed dataset in smaller, user-controlled segments. The function allows users to select the number of rows displayed per page (10, 25, 50, or 100) and separates header rows from data rows, allowing for headers to be conveniently seen in every page. Pagination logic computes the total number of pages based on the selected page size and dynamically determines which subset of rows to display using index-based slicing. Navigation controls ("Previous" and "Next") are provided to allow sequential traversal through the dataset, with the current page state stored using Streamlit's session state. This approach prevents excessive scrolling, reduces visual clutter, and avoids performance degradation that can occur when rendering large tables in a single view.

In addition to pagination, data caching is used to optimize performance and reduce redundant file reads. The dashboard uses Streamlit's @st.cache_data decorator to cache the output of the custom CSV loading function. Once a dataset is parsed and loaded into memory, subsequent query interactions reuse the cached data rather than re-reading the file from disk. This significantly reduces I/O overhead and improves application responsiveness, particularly when repeatedly operating on large datasets.

4.0 Learning Experience and Conclusions

This project bridged low-level data-processing implementation with high-level analytical workflows. By manually constructing each operation and integrating them into a functioning application, we gained not only a deeper understanding of how data management libraries operate internally, but also a deeper appreciation for how these core concepts translate into quick, practical tools for data exploration.

One key learning outcome from this project was an increased awareness of data quality and real-world complexity. In practice, datasets rarely adhere to idealized formats, and robust preprocessing is often just as critical as the analytical methods applied to them. Prior to executing any analysis functions, the application was required to handle a range of inconsistencies, including mixed numeric formats (such as "48,000" and "48,0"), missing values (represented as "<NA>" or empty strings), and textual fields containing embedded commas (such as "Total coffee, tea, and spices"). Early in development, these irregularities were a frequent source of runtime errors and unexpected behavior, requiring careful debugging to identify their origins. Through iterative refinement of parsing, cleaning, and normalization logic, these issues were resolved, ultimately resulting in a stable and reliable application.

Another important learning experience involved collaboration and project structure. Our team utilized a shared GitHub repository to ensure that all development progress was consistently tracked and safely merged into the main branch. To avoid duplicated work and maintain alignment, we established a practice of communicating summaries of changes following each commit. As a two-person team, we also had to thoughtfully consider scalability constraints and select appropriate strategies given our time and resource limitations. Drawing on course concepts such as vertical versus horizontal scaling, PySpark, and chunk-based processing, we determined that chunked reading and filtering were the most practical and effective solutions for this project based on time and resource availability.

In conclusion, this project allowed us to explore the fundamental mechanisms underlying common data processing operations while translating theoretical concepts into a practical, interactive application. By implementing core SQL functionalities and integrating scalability considerations without relying on high-level data processing libraries, we developed a deeper understanding of how these operations are executed at a lower level. Overall, the project reinforced the importance of thoughtful system design, data preprocessing, functional programming, and modular development while highlighting how data management principles can be leveraged to build efficient analytical tools. Potential avenues for future work include enabling user-uploaded datasets, supporting additional file formats, allowing users to download processed data for external analysis, and improving scalability by extending chunk-based processing to remaining data operations such as sorting, grouping, aggregation, and joins.

5.0 Project Work Split

	Contributions
Viviana Alba	Parsing, Projection, Filtering, Sorting, Group and Aggregate, Limit, Table Pagination, Dashboard UI and Complete Pipeline Execution, Project Report Writing, Demo Presenter
Kayla Hoffman	Parsing, Read CSV and Chunked Reading, Join, Chunked Filtering, Group and Aggregate, Dashboard UI and Complete Pipeline Execution, Project Report Writing, Demo Presenter

Table 1.0 Team Member Contributions and Task Split

References

[1] Elgiriyewithana, N. (2023). *Global country information dataset 2023* [Data set]. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/6101670

[2] HackerRank. Quicksort 2 – Sorting. https://www.hackerrank.com/challenges/quicksort2/problem

[3] U.S. Department of Agriculture, Economic Research Service. (2025). *U.S. food imports data* [Data set]. https://www.ers.usda.gov/data-products/us-food-imports